Relatório – TP1

Serviços de Monitorização Urbana para One Health

Repositório: https://github.com/AfonsoSilva14/SistemasDistribuidos.git

1. Introdução

O presente trabalho propõe um sistema distribuído de monitorização ambiental urbana enquadrado no paradigma One Health, que reconhece a interdependência entre saúde humana, animal e ambiental. O sistema recolhe variáveis ambientais como qualidade do ar, temperatura, humidade e ruído em zonas urbanas, permitindo identificar padrões e potenciais riscos para a saúde pública de forma contínua e automatizada.

A solução é composta por três entidades — SENSOR, GATEWAY e SERVIDOR — que comunicam através de um protocolo textual definido de raiz, implementado sobre sockets TCP em C#. A arquitetura foi concebida para garantir fiabilidade nas trocas de mensagens, validação intermédia dos dados pelo gateway e armazenamento persistente no servidor. O sistema suporta múltiplos sensores em simultâneo e tolera falhas de sensores individuais sem comprometer o funcionamento global.

2. Arquitetura do Sistema

O sistema segue uma arquitetura distribuída em três camadas com responsabilidades bem delimitadas, o que facilita a manutenção, a escalabilidade e o isolamento de falhas:

- **SENSOR:** recolhe dados ambientais, autentica-se no gateway, regista os tipos de dados que suporta e envia medições. Emite heartbeats automáticos a cada 30 segundos para sinalizar presença ativa e disponibiliza um menu interativo para operação manual.
- **GATEWAY:** entidade central de validação e encaminhamento. Autêntica cada sensor contra um registo local, verifica a coerência dos dados recebidos (tipo, valor e unidade) e encaminha apenas dados válidos para o servidor. Monitoriza a atividade dos sensores por heartbeat e agrega métricas localmente.
- **SERVIDOR:** entidade de armazenamento persistente. Recebe dados já validados do gateway, organiza-os em ficheiros por tipo de medição e numa base de dados SQLite, permitindo consultas posteriores por variável ambiental ou por sensor.

A separação clara entre validação (gateway) e armazenamento (servidor) é uma decisão deliberada: o servidor recebe apenas dados já verificados, o que simplifica a sua implementação e garante a qualidade do que é persistido. A comunicação entre entidades é feita por sockets TCP, que asseguram entrega ordenada e fiável das mensagens. O diagrama de sequência completo encontra-se no Anexo A.

3. Protocolo de Comunicação

O protocolo foi desenvolvido de raiz com um formato textual simples: cada mensagem ocupa uma linha, com campos separados pelo carácter. Esta escolha justifica-se por três razões: (1) legibilidade e facilidade de depuração durante o desenvolvimento; (2) compatibilidade direta com StreamReader/StreamWriter em C#, sem necessidade de serialização binária; (3) simulação manual trivial via terminal. O formato geral é:

```
TIPO_MENSAGEM|campo1|campo2|...
```

O protocolo organiza-se em dois canais independentes — SENSOR↔GATEWAY e GATEWAY↔SERVIDOR — cada um com o seu conjunto de mensagens e respostas. Todos os comandos têm uma resposta explícita de confirmação (ACK) ou rejeição (NACK/FAIL) com indicação do motivo específico — por exemplo, DATA_NACK|S102|timestamp|INVALID_UNIT. Esta abordagem garante que o emissor sabe sempre se a mensagem foi aceite ou rejeitada e porquê, permitindo-lhe reagir de forma inteligente sem ambiguidade.

Cada entidade possui um ciclo de vida definido por estados (ex: SENSOR: DISCONNECTED → CONNECTED → IDENTIFIED → REGISTERED → ACTIVE → CLOSING). Estes estados garantem que mensagens fora de sequência são rejeitadas — por exemplo, o gateway rejeita DATA de um sensor que ainda não completou o REGISTER_TYPES com o código NOT_READY, evitando que dados cheguem sem que o gateway saiba que tipos esperar. A especificação completa das mensagens e estados encontra-se no Anexo B.

Canal	Mensagens Enviadas	Respostas Possíveis
SENSOR → GW	HELLO, REGISTER_TYPES, DATA, HEARTBEAT, BYE	HELLO_ACK, AUTH_OK/FAIL, REGISTER_ACK/NACK, DATA_ACK/NACK, HEARTBEAT_ACK, BYE_ACK
GW → SERVIDOR	GW_HELLO, GW_DATA, GW_HEARTBEAT, GW_BYE	GW_HELLO_ACK, SERVER_ACK/NACK, GW_HEARTBEAT_ACK, GW_BYE_ACK

4. Implementação

4.1 Modelo de Concorrência e Tratamento de Erros

O gateway é implementado como um servidor TCP com um loop de aceitação (AcceptTcpClient) no thread principal. Cada sensor ligado é tratado numa Task independente (Task.Run), o que permite atender múltiplos sensores em simultâneo sem criar threads do sistema operativo desnecessariamente — o runtime do .NET gere um pool de threads partilhado, mais eficiente para cargas variáveis. O mesmo modelo aplica-se ao servidor para múltiplos gateways. Uma Task adicional em background monitoriza os heartbeats dos sensores a cada 30 segundos.

Cada handler de sensor está envolvido num bloco try/catch/finally que garante o fecho limpo da ligação TCP mesmo em caso de erro de rede ou exceção não tratada — sem este mecanismo, ligações encerradas abruptamente poderiam deixar recursos abertos indefinidamente. Esta arquitetura assíncrona implica acesso concorrente a recursos partilhados, o que obrigou à definição de múltiplos locks granulares — um por recurso — para minimizar contenção e evitar deadlocks:

Lock (Gateway)	Recurso Protegido
sensorDbLock	Dicionário em memória com o estado e LastSync dos sensores
sensoresFileLock	Ficheiro sensores.txt — persistência do estado em disco
dadosRecebidosFileLock	Ficheiro dados_recebidos.txt — registo cronológico de medições
agregadoFileLock	Ficheiro agregado.txt — médias acumuladas por sensor/tipo
serverConnectionLock	Stream TCP partilhado entre todas as tasks e o servidor

A opção por locks granulares (um por ficheiro) em vez de um lock global garante que, por exemplo, a escrita em dados_recebidos.txt não bloqueia simultaneamente a escrita em agregado.txt, reduzindo a latência em cenários com múltiplos sensores ativos. No servidor, usa-se um ConcurrentDictionary<string, object> para criar dinamicamente um lock por tipo de dado, evitando que medições de TEMP e PM25 se bloqueiem mutuamente ao escrever em ficheiros distintos.

4.2 Cadeia de Validações no Gateway

Antes de aceitar qualquer medição, o gateway aplica sequencialmente as seguintes verificações, respondendo com um NACK específico em cada passo. A sequencialidade é intencional: uma falha precoce evita processamento desnecessário nos passos seguintes.

- **Autenticação:** sensor registado no ficheiro de configuração e com estado ativo — rejeita manutenção (MAINTENANCE) e desativado (DISABLED)
- **Zona:** zona declarada pelo sensor coincide com a zona registada, evitando spoofing de localização (INVALID_ZONE)
- **Sequência:** REGISTER_TYPES obrigatório antes de qualquer DATA; garante que o gateway conhece os tipos esperados (NOT_READY)
- **Tipo:** tipo de dado presente na lista registada; normalização automática PM2.5 → PM25 (UNSUPPORTED_TYPE)
- **Valor:** número válido em formato pt-PT ou invariant culture — suporta vírgula e ponto decimal (INVALID_VALUE)
- **Unidade:** unidade compatível com o tipo (ex: TEMP aceita °C; PM25 aceita µg/m³; CO2 aceita ppm) (INVALID_UNIT)

5. Funcionalidades Extra

5.1 Persistência com SQLite

Optou-se por SQLite (Microsoft.Data.Sqlite) em vez de um servidor de base de dados externo porque: não requer processo separado nem configuração de rede, simplificando o arranque em qualquer máquina; a base de dados é um ficheiro único e portável (.db), adequado à escala deste projeto; e suporta transações ACID, garantindo integridade dos dados mesmo em caso de falha durante a escrita. O gateway mantém gateway.db com as tabelas Sensores (estado atual de cada sensor, incluindo LastSync) e MedicoesGateway (histórico completo de todas as medições recebidas). O servidor mantém servidor.db com a tabela MedicoesServidor, que replica o histórico acrescentando o gateway de origem — permitindo auditoria cruzada entre as duas bases de dados.

5.2 Deteção Automática de Sensores Inativos

Uma Task em background executa no gateway a cada 30 segundos e verifica o campo LastSync de cada sensor ativo. Se o tempo decorrido desde o último heartbeat exceder 60 segundos — o dobro do intervalo de heartbeat, margem intencional para tolerar um heartbeat perdido por congestionamento de rede sem penalizar o sensor prematuramente — o sensor é automaticamente marcado como manutenção. O estado é atualizado em memória, no ficheiro sensores.txt e na base de dados gateway.db, garantindo que a deteção persiste mesmo após reinício do gateway e que sensores com falha de rede são rejeitados em ligações futuras sem intervenção manual.

5.3 Agregação Local de Dados no Gateway

O gateway mantém o ficheiro agregado.txt com a contagem e média acumulada de cada tipo de medição por sensor. A cada nova medição válida, a média é recalculada de forma incremental — somando ao acumulador e incrementando o contador — sem necessidade de reler o histórico completo, o que torna a operação independentemente do volume de dados acumulado. Este mecanismo permite consultar tendências localmente mesmo sem acesso ao servidor, com o formato: sensorId|tipo|quantidade|media.

5.4 Heartbeat Automático no Sensor e Organização no Servidor

O sensor executa uma Task em background que emite automaticamente HEARTBEAT a cada 30 segundos após autenticação bem-sucedida, em paralelo com o menu interativo. O acesso ao stream TCP partilhado é protegido por um networkLock que serializa os acessos entre o utilizador e o heartbeat automático, evitando intercalação de mensagens que corromperia o protocolo. No servidor, os dados são organizados em ficheiros individuais por tipo de medição (ex: dados_TEMP.txt, dados_PM25.txt), facilitando análises por variável ambiental sem necessidade de filtrar um ficheiro único de registo geral.

6. Conclusão

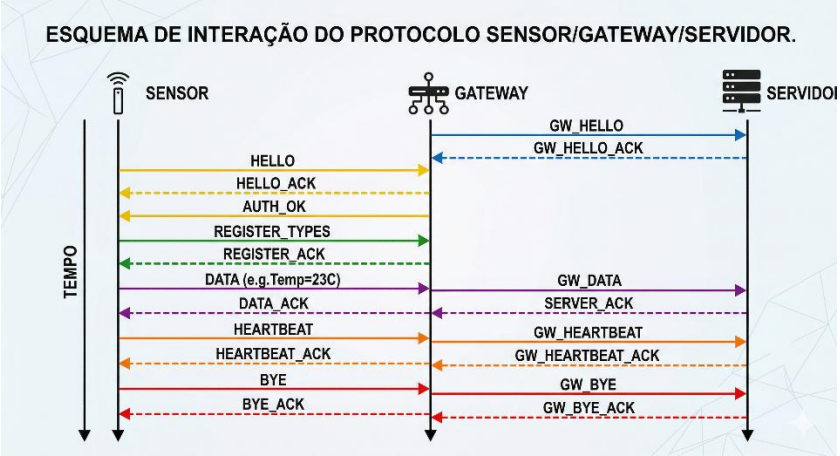
O sistema desenvolvido aplica de forma coerente os conceitos fundamentais de sistemas distribuídos: protocolo próprio com confirmação explícita de cada mensagem, concorrência gerida por Tasks com locks granulares, validação centralizada no gateway e persistência híbrida em ficheiros estruturados e SQLite. A decisão de concentrar toda a validação no gateway — e não repartir essa responsabilidade pelo servidor — simplifica a arquitetura e garante que apenas dados íntegros são persistidos, independentemente do número de sensores ou do volume de medições.

As funcionalidades extra implementadas — deteção automática de inativos com margem de tolerância de dois intervalos de heartbeat, agregação incremental, heartbeat automático com proteção de concorrência, e organização do servidor por tipo de dado — conferem robustez operacional sem introduzir dependências externas além do .NET.

ANEXOS

Anexo A – Diagrama de Sequência (SENSOR / GATEWAY / SERVIDOR)

O diagrama representa o fluxo completo de interações entre as três entidades, com a sequência temporal das mensagens trocadas. Setas a cheio: mensagens enviadas; setas a tracejado: respostas (acknowledgements).



Anexo B – Especificação Completa do Protocolo

Mensagens SENSOR → GATEWAY:

```
HELLO|sensor_id|zona
REGISTER_TYPES|sensor_id|tipo1,tipo2,...
DATA|sensor_id|timestamp|tipo|valor|unidade
HEARTBEAT|sensor_id|timestamp
BYE|sensor_id|timestamp
```

Respostas GATEWAY → SENSOR:

```
HELLO_ACK|gateway_id|OK
AUTH_OK|sensor_id|ativo|zona  /
AUTH_FAIL|sensor_id|{NOT_REGISTERED|MAINTENANCE|DISABLED|INVALID_ZONE}
REGISTER_ACK|sensor_id|OK|tipos  /  REGISTER_NACK|sensor_id|INVALID_TYPES|tipos_invalidos
DATA_ACK|sensor_id|timestamp|OK  /
DATA_NACK|sensor_id|timestamp|{UNSUPPORTED_TYPE|INVALID_VALUE|INVALID_UNIT|...}
HEARTBEAT_ACK|sensor_id|timestamp|OK
BYE_ACK|sensor_id|timestamp|OK
```

Mensagens GATEWAY → SERVIDOR:

```
GW_HELLO|gateway_id|ip_gateway
GW_DATA|gateway_id|sensor_id|zona|timestamp|tipo|valor|unidade
GW_HEARTBEAT|gateway_id|timestamp
GW_BYE|gateway_id|timestamp
```

Respostas SERVIDOR → GATEWAY:

```
GW_HELLO_ACK|gateway_id|OK
SERVER_ACK|gateway_id|sensor_id|timestamp|OK  /  SERVER_NACK|...|motivo
GW_HEARTBEAT_ACK|gateway_id|timestamp|OK
GW_BYE_ACK|gateway_id|timestamp|OK
```

Anexo C – Exemplo de Diálogo Completo

SENSOR ↔ GATEWAY:

```
HELLO|S102|ZONA_ESCOLAR → HELLO_ACK|GW01|OK
AUTH_OK|S102|ativo|ZONA_ESCOLAR
REGISTER_TYPES|S102|TEMP,PM25 → REGISTER_ACK|S102|OK|TEMP,PM25
DATA|S102|2026-03-10T09:15:00|PM25|78|ug/m3 → DATA_ACK|S102|2026-03-10T09:15:00|OK
HEARTBEAT|S102|2026-03-10T09:15:10 → HEARTBEAT_ACK|S102|2026-03-10T09:15:10|OK
BYE|S102|2026-03-10T09:20:00  → BYE_ACK|S102|2026-03-10T09:20:00|OK
```

GATEWAY ↔ SERVIDOR:

```
GW_HELLO|GW01|192.168.1.10 → GW_HELLO_ACK|GW01|OK
GW_DATA|GW01|S102|ZONA_ESCOLAR|2026-03-10T09:15:00|PM25|78.00|ug/m3  →
SERVER_ACK|GW01|S102|2026-03-10T09:15:00|OK
GW_BYE|GW01|2026-03-10T09:20:05 → GW_BYE_ACK|GW01|2026-03-10T09:20:05|OK
```

Anexo D – Estrutura de Ficheiros e Base de Dados

Entidade	Ficheiro / BD	Conteúdo
Gateway	sensores.txt	sensor_id;estado;zona;tipos;last_sync — estado persistente dos sensores
Gateway	dados_recebidos.txt	timestamp|sensorId|zona|tipo|valor|unidade — registo cronológico
Gateway	agregado.txt	sensorId|tipo|quantidade|media — médias acumuladas (cálculo incremental O(1))
Gateway	gateway.db	Tabelas: Sensores (estado atual) e MedicoesGateway (histórico completo)
Servidor	dados_TIPO.txt	Ex: dados_TEMP.txt, dados_PM25.txt — dados organizados por variável
Servidor	servidor.db	Tabela MedicoesServidor (gatewayId, sensorId, zona, timestamp, tipo, valor, unidade)

Anexo E – Estados do Sistema

Entidade	Ciclo de Estados
SENSOR	DISCONNECTED → CONNECTED → IDENTIFIED → REGISTERED → ACTIVE → CLOSING → CLOSED
GATEWAY	WAIT_CONNECTION → WAIT_HELLO → VALIDATING → READY / REJECTED → MONITORING → FORWARDING → CLOSED
SERVIDOR	WAIT_GATEWAY → RECEIVING → STORING → ACK → CLOSED